

Comprehensive Analysis of Autonomous Agent Architectures in the Screenshot Environment: From TooAngel to Contemporary Hiveminds

The emergence of the Screenshot environment as a persistent, massive multiplayer online real-time strategy (MMORTS) platform for programmers has necessitated the development of increasingly sophisticated autonomous agents. At the heart of this technical evolution is the TooAngel codebase, a project that is widely recognized as the first fully automated open-source bot to successfully navigate the complex interplay of resource management, base construction, and inter-player diplomacy.¹ To understand the significance of TooAngel, one must first recognize the fundamental constraints of the Screenshot server-side architecture. The platform operates on a Node.js 8.9.3 backend utilizing MongoDB and Redis to manage over 70,000 interconnected rooms in a persistent world.³ Player scripts are executed within isolated virtual machine contexts, subject to strict CPU limits that correlate with the player's Global Control Level (GCL).³ Within this rigorous computational economy, TooAngel introduced a paradigm of "Full Automation," shifting the developer's role from a manual commander to an architect of emergent systems.¹ This analysis explores the architectural underpinnings of TooAngel, contrasts its design with subsequent paradigms like Overmind and BonzAI, and evaluates the technical strengths and weaknesses that define the current state of autonomous agent development.

The Foundations of TooAngel Architecture

The TooAngel codebase is built upon a philosophy of complete self-sufficiency. It was designed not merely to assist a player, but to function as a fully autonomous NPC (non-player character) capable of reaching Room Control Level 8 (RCL 8) without human intervention.¹ This architectural goal required a robust framework for handling every game mechanic, from basic energy harvesting to complex mineral synthesis and market trading.

BrainController and the MVC Pattern

Internal logic in the TooAngel bot follows a variant of the Model-View-Controller (MVC) framework, centered around a main execution loop in `main.js` that instantiates the `BrainController`.¹⁰ The `BrainController` serves as the primary orchestrator, dividing each game tick into two critical phases: stimuli processing and action execution.¹⁰ During the stimuli phase, the bot scans the environment to update its mental model of the world, verifying the integrity of the Memory object and detecting state changes such as hostile incursions or the depletion

of resource sources.¹⁰ This phase is computationally intensive, as it often involves iterating over numerous game objects across multiple rooms.

Following the assessment of environmental stimuli, the action phase delegates control to lower-level managers, including the Room Controller, Job Controller, and Creep Controller.¹⁰ These sub-systems translate high-level strategic goals into specific intents, which are the 0.2 CPU actions that actually modify the game state.⁶ This hierarchical separation ensures that the bot can maintain a consistent strategic direction while allowing individual units—referred to as "creeps"—to execute tactical behaviors based on their assigned roles.

Evolutionary Mechanics through World-Driven Development

TooAngel pioneered a unique socio-technical development model known as "World Driven" integration.¹ This paradigm involves an automated continuous integration and deployment (CI/CD) pipeline where community pull requests are automatically merged and deployed to the bot's active account on the public Screenshot server.¹ This creates a Darwinian feedback loop where the codebase evolves based on real-world performance against human opponents and rival bots. The bot's reputation and survival in the persistent world provide the primary metrics for code success, representing a shift from static software development to a living, reactive agent that learns through the collective input of the Screenshot community.¹

TooAngel Development Feature	Mechanism	Objective
World Driven Merging	Automated PR integration ¹	Direct evolutionary feedback from gameplay ¹
Full Automation	No-interface control ⁹	Comprehensive self-sufficiency ²
NPC Deployment	Public and private server presence ¹	Benchmarking and community challenge ¹²
Documentation	GitHub Pages / Markdown ²	Contributor onboarding and transparency ¹

Base Planning and Spatial Logic in Autonomous Environments

The physical layout of a colony in Screenshot is one of the most critical factors determining long-term efficiency. Inefficient placement of structures increases fatigue and travel time for

creeps, leading to a significant loss in energy throughput over thousands of ticks. TooAngel addresses this through a path-driven algorithmic approach to room layout.

Path-Anchored Structure Placement

Unlike manual players who might place buildings based on aesthetic or defensive intuition, TooAngel's base building logic is entirely derived from pathing efficiency.¹⁴ The process begins with the identification of a pathStart position, which is typically located adjacent to the room's primary storage structure and the controller upgrader.¹⁴ From this origin, the bot precalculates paths to every source of energy, the mineral deposit, and the midpoints of each room exit.¹⁴

Structures are then placed along these precalculated spines. Spawns, extensions, labs, and towers are positioned in a way that minimizes the distance a creep must travel from the central logistics hub.¹⁴ This system relies on cached paths to reduce the CPU cost of pathfinding during unit movement, as most creeps are restricted to these pre-defined routes.¹⁴ While this reduces computational overhead, it introduces a vulnerability: the bot's movements become highly predictable, allowing experienced human players to block or "trap" the bot by positioning stationary units along these fixed paths.¹

Comparison of Modern Base Planning Paradigms

Contemporary autonomous agents have moved toward more complex spatial analysis techniques, such as Distance Transforms and Floodfill algorithms, to improve upon the path-anchored model used by TooAngel.

- **Distance Transform Algorithms:** These are used to identify the most open areas of a room by calculating the distance of every walkable tile from the nearest wall.¹⁵ This is particularly useful for "bunker" designs where structures are packed into a tight 11x11 square to minimize the rampart area requiring repair.¹⁵
- **Floodfill Accessibility:** This method categorizes tiles based on their travel distance from the room's core. It ensures that high-traffic buildings, such as extension networks, are placed in the most accessible locations.¹⁵
- **Min-Cut Rampart Placement:** Modern bots often use the min-cut algorithm to find the optimal line of ramparts that will defend a set of structures with the minimum number of tiles, thereby reducing the massive energy upkeep associated with rampart decay.¹⁵

Planning Methodology	Core Algorithm	Structural Outcome
Path-Driven (TooAngel)	A* Precalculation ¹⁴	Spine-based layouts along traffic routes ¹⁴

Bunker (Overmind)	Distance Transform ¹⁶	Tightly packed 11x11 core under ramparts ¹⁶
Grid (International Bot)	Fixed Coordinate Offsets ¹⁸	Modular, repeatable structure groups ¹⁸
Adaptive (BonzAI)	POI-based building ¹⁹	Context-aware placement around sources ¹⁹

Resource Logistics and Unit Management Strategies

The management of units—known as "creeps"—is the fundamental tactical layer of any Screeps bot. The efficiency with which a codebase handles the lifecycle of these units, from spawning to task assignment and eventually death, determines the bot's ability to expand and compete.

Role-Based Specialization in TooAngel

TooAngel utilizes a specialized role system where each creep is assigned a discrete set of behaviors upon spawning.²⁰ These roles are optimized for specific segments of the economy:

- **Sourcers:** Units dedicated to harvesting energy from sources and depositing it into containers or links.²⁰
- **Carry Units:** Dedicated haulers that move resources between containers, storage, and consumers like extensions or towers.²⁰
- **Upgraders:** Stationary or semi-stationary units that pump energy into the Room Controller to advance GCL and RCL.¹⁴
- **Builders and Repairers:** General maintenance units that construct new structures and keep existing ones from decaying.²⁰

This specialization allows for efficient "Part-Count Balancing," where the bot calculates exactly how many WORK or CARRY parts are needed to maximize a room's potential, rather than simply counting the number of units.²¹ However, a weakness of this static role assignment is that units cannot easily adapt to sudden changes in environmental priorities unless explicitly re-tasked by a high-level manager.²²

The Shift to Swarm Intelligence and Mission-Based Frameworks

Modern codebases like Overmind and BonzAI have moved away from simple role assignments toward more flexible abstractions. Overmind employs a hierarchical swarm intelligence model based on "Overlords" and "Directives".²³ An Overlord is a conceptual manager of a specific goal—such as bootstrapping a new colony or mining a remote source—that dynamically spawns and directs "Zerg" units as needed.²³

BonzAI utilizes a "Mission and Operation" framework.²⁵ In this paradigm, an Operation defines a

high-level strategic objective (e.g., a "MiningOperation" for a remote room), while Missions define the specific tactics and units required to achieve it.²⁵ This modularity allows for the reuse of complex behaviors across different strategic contexts, a level of abstraction that the more prototype-heavy TooAngel codebase lacks.²³

Computational Efficiency and CPU Resource Management

In Screamers, CPU is the ultimate currency. Every successful bot must carefully manage its computational budget to avoid triggering script timeouts, which can result in a total loss of a game tick.³

Bucket Management and Intent Grouping

The Screamers engine uses a "bucket" system to allow for uneven CPU usage. Players have a baseline limit (e.g., 30 CPU + 10 per GCL) and can accumulate up to 10,000 unused CPU units in a reservoir.⁶ This bucket can be drawn upon to execute bursts of intensive work, such as pathfinding across multiple rooms or processing market data.⁶

TooAngel manages this budget by minimizing the frequency of expensive searches. By caching paths and structure layouts, the bot reduces its per-tick CPU floor.¹³ Furthermore, high-level managers can "group" intents to save resources. For example, a miner sitting on a container can harvest energy that drops directly into the container, avoiding the 0.2 CPU cost of an explicit transfer action every tick.²⁸

The Overmind "Refresh" Mechanism

A significant innovation in the Overmind architecture is the "refresh" cycle designed to mitigate the overhead of object-oriented programming in JavaScript.³⁰ Classes are expensive to instantiate in the V8 engine, especially when shared references to game objects are reconstructed every tick.³⁰ Overmind addresses this by running a full build() phase every 20 ticks to re-instantiate its script objects, while using a lighter refresh() phase on all other ticks to update references to the current game state without destroying existing class instances.³⁰ This "soft update" significantly lowers garbage collection pressure and increases the complexity of the logic the bot can execute within its limit.³⁰

The Emergence of Operating System Paradigms

Advanced Screamers bots have increasingly adopted an "Operating System" (OS) paradigm to manage execution order and CPU allocation.¹⁷ In an OS bot, game logic is divided into "processes" managed by a kernel and a scheduler.³²

- **Process Isolation:** Each task, such as a room management loop or a long-distance scout

mission, runs as an independent process with its own ID and priority.³²

- **Prioritized Scheduling:** The scheduler allocates a slice of the per-tick CPU budget to each process based on its importance. Critical defense processes run every tick, while low-priority infrastructure planning might only run every 10 or 100 ticks.¹⁷
- **Decoupling from Ticks:** An OS bot can stretch or compress its workload. If the CPU bucket is low, the scheduler might spread a 250 CPU task over five ticks, ensuring that essential functions continue to run without crashing the script.³³

Memory Management and State Persistence

Because the entire player script is re-run from scratch every game tick, state must be persisted using the Memory object, a global JSON container limited to 2,048 KB.³⁴

Serialization and the 2MB Constraint

The Memory object is serialized and deserialized every tick, which consumes a portion of the player's CPU limit based on the size and complexity of the stored data.³ Storing full game objects is impossible, as they contain methods and circular references that cannot be stringified; instead, bots must store object IDs and resolve them each tick using `Game.getObjectById()`.³⁶

TooAngel and other mature codebases employ "RawMemory Segments" to bypass the 2MB limit. The platform provides 100 segments of 100KB each, allowing bots to shard their data.³⁵ A bot might store its room map data in one segment, its market history in another, and its inter-player reputation ledger in a third, only activating the segments it needs for a specific tick.³⁵

Caching and MemHack

To avoid the cost of deserializing the entire Memory object every tick, many high-level bots use a community-developed technique known as "MemHack." This involves storing a reference to the parsed memory object in the global scope, allowing the bot to bypass the engine's standard deserialization process as long as the underlying VM context persists.³⁹ This is critical for bots like Overmind that rely on complex, deeply nested data structures.³⁰

Diplomatic Systems and Social AI

Screeps is not just a game of resource management but also of social dynamics. Bots that operate on the public server must navigate alliances, respect territorial boundaries, and respond to aggression.

Reputation and Retaliation in TooAngel

TooAngel features one of the most well-documented diplomatic modules in the open-source

community.¹ The system functions by tracking player actions and assigning a reputation score based on whether an interaction was friendly or hostile.¹ This information is stored in persistent memory and shared across all of the bot's controlled rooms.¹

A key component of this system is "Trapped Scenario Detection".¹ This module identifies when a human player is attempting to "peacefully" disable the bot by surrounding its spawn or blocking its exits without initiating combat. When such a scenario is detected, the bot can escalate its DEFCON level and trigger specific escape or counter-offensive routines.¹

The Overmind Assimilator and Collective Intelligence

A more radical approach to inter-player interaction is seen in the Overmind "Assimilator".²³ This system allows multiple independent players running the Overmind script to act as a single, unified hivemind.²³

- **Shared Intelligence:** Assimilated players share a master ledger of directives, allowing them to coordinate attacks or defensive efforts across vast distances.²³
- **Resource Redistribution:** The system automatically identifies when one member of the hivemind is low on energy and directs others to send resources via the terminal network.²³
- **Verification and Security:** To prevent "impostor" bots from joining the collective to steal resources, the Assimilator uses a cryptographic hash system.³⁰ Parts of the codebase are marked with @assimilationLocked decorators; if a player modifies these core modules, their hash will not match the master ledger, and they will be excluded from the hive.³⁰

Detailed Comparison of Major Codebases

The following table synthesizes the architectural and philosophical differences between the primary paradigms explored in this analysis.

Paradigm Attribute	TooAngel	Overmind	BonzAI / Quorum	Choreographer
Philosophy	Full automation through prototypes ¹	Hierarchical swarm (Zerg-themed) ²³	Extensible Mission framework ²⁵	Event-driven Operating System ¹⁷
Data Integrity	Manual memory checks ¹⁰	Cryptographic codebase hashing ³⁰	Unique controller signatures ⁴¹	Scribe-persisted game state ¹⁷

Unit Management	Role-based specialization ²⁰	Directive-driven overlords ²⁴	Abstract Operation/Mission ²⁵	Scheduler-managed threads ¹⁷
Social Stance	Diplomatic reputation tracking ²	Universal Assimilator Hivemind ²³	Strict neutrality (no alliances) ⁴¹	Auto-attack of weaker players ¹⁷
Language	Plain JavaScript ¹	Typed TypeScript ²³	Typed TypeScript ⁴²	Modern TypeScript/JS ¹⁷
Base Building	Precalculated traffic spines ¹⁴	Modular bunkers and ramparts ¹⁶	Adaptive proximity placement ¹⁹	Min-cut optimized defense ¹⁷

Technical Strengths and Weaknesses

Evaluating TooAngel in the context of its modern successors reveals a codebase that pioneered many essential concepts but struggles with the technical debt of an aging architecture.

Strengths of the TooAngel Paradigm

The primary strength of TooAngel is its remarkably low computational floor. Because it avoids the overhead of deep class hierarchies and relies on cached, spine-based pathing, it can operate efficiently in environments with severe CPU constraints.¹⁴ This makes it an ideal codebase for private servers or for players just starting their GCL progression.⁸

Furthermore, its "World Driven" development history has resulted in a set of extremely robust, battle-tested algorithms for base recovery.¹ TooAngel is famously resilient; even if a room is completely leveled by an attacker, the bot's ability to automatically rebuild from a single spawn and reclaim its pre-optimized layout is among the best in the Screenshot community.¹

Weaknesses and Legacy Constraints

The reliance on plain JavaScript and an older prototype-based structure is a significant long-term disadvantage. Modern codebases benefit from the static typing and smart refactoring tools provided by TypeScript, which prevents an estimated 15% of common logic bugs.²³ TooAngel's structure can be "cryptic" and difficult for new developers to modify compared to the clean, mission-based abstractions of BonzAI.¹⁰

Additionally, the predictability of TooAngel's path-anchored logic makes it highly exploitable. High-level players often view TooAngel-based bots as "lazy" or "NPC-tier" opponents that can be countered through simple physical obstruction or the exploitation of its reputation logic.⁴⁷

Without a more dynamic pathfinding system that accounts for moving obstacles and destructible walls, the bot is often "stuck" in suboptimal movement loops that human-controlled agents easily bypass.³¹

Theoretical Optimization and the Mathematical Limit of Control

The progression of an autonomous agent in Screamers can be modeled as a series of optimization problems across spatial, temporal, and social dimensions.

GCL and the CPU Ceiling

A player's progression is primarily governed by the Global Control Level (GCL), which is advanced by pumping energy into room controllers. The energy required to reach the next level is defined by the default formula:

$$RequiredPoints = (GCL - 1)^{2.4} \times 1,000,000$$

This power-law progression means that expansion becomes exponentially more difficult, requiring bots to achieve near-perfect efficiency in energy harvesting and distribution to continue growing.⁷ Higher GCL increases the CPU limit (up to a maximum of 300 on public servers), setting the technical "complexity ceiling" for the agent's logic.⁵⁰

Temporal Efficiency and Tick Optimization

Because a tick only ends when all player scripts have finished executing, the duration of a tick is variable, typically ranging from 3 to 5 seconds.³ A bot that consistently uses its full 500ms allotment of CPU per tick is effectively "renting" a larger share of the server's Intel Xeon processor cores, allowing it to perform deeper searches and more complex logistical balancing.³ This temporal efficiency is the core advantage of OS-style codebases like Choreographer, which can maintain a consistent workload even when the game world triggers massive events like global resets.¹⁷

Socio-Technical Implications of Open-Source Bot purges

The widespread availability of powerful open-source bots has led to an interesting socio-technical phenomenon in Screamers: the "bot purge." Groups of experienced players often dedicate themselves to hunting down and eliminating bots like Overmind or TooAngel.⁴¹

This behavior is driven by two factors. First, many players believe that running someone else's

code undermines the "personal challenge" ethos of the game.⁴¹ Second, open-source bots provide a predictable target for testing advanced combat logic. Because these bots have known structure and signing signatures on their controllers, they can be easily identified by automated scout units.⁴¹ This has forced bot developers to include features like unique controller signatures and varied creep naming conventions to evade detection, adding a layer of "digital camouflage" to the architectural requirements of an autonomous agent.⁴¹

The Future Landscape of Computational Strategy

The trajectory of the Screenshot platform and its autonomous agents suggests a move toward even greater abstraction and performance.

WebAssembly and Rust Integration

To overcome the performance limitations of the V8 JavaScript engine, some players have begun rewriting their core logic—specifically pathfinding and matrix convolutions—in Rust or C++, which is then compiled to WebAssembly (WASM).⁵ WASM provides a significant increase in execution speed, allowing bots to perform hundreds of thousands of operations per tick that were previously impossible in JavaScript.⁵ This trend signals a future where the most competitive bots are no longer written in high-level scripting languages but are high-performance computational engines.

Real-Time Telemetry and Empire Dashboards

The integration of external monitoring tools has become a standard part of the Screenshot architecture. Many bots now include agents that collect statistics on energy production, market trades, and combat metrics, pushing this data to external Grafana dashboards.¹³ This provides developers with real-time telemetry, allowing them to identify CPU bottlenecks or logistical failures that are not obvious from the in-game interface.¹³

Conclusions

The TooAngel codebase remains a seminal work in the history of autonomous agents, representing the transition from manual unit scripts to a comprehensive philosophy of full automation and community-driven evolution. Its architecture, characterized by prototype-based MVC logic and path-anchored base planning, established the foundational requirements for successful self-sufficiency in the Screenshot environment. However, as the computational and social complexity of the game has grown, the limitations of this paradigm—specifically its rigidity and technical debt—have paved the way for more advanced models.

Modern agents have evolved toward swarm intelligence, mission-based frameworks, and prioritize prioritized process scheduling. These innovations, seen in codebases like Overmind and Choreographer, allow for greater scalability and adaptability in a 24/7 competitive

environment. The shift toward collective hiveminds and the integration of high-performance technologies like WebAssembly further indicate that the future of autonomous computational strategy lies in a convergence of computer science, decentralized systems, and social AI. Ultimately, the legacy of TooAngel is not just its code, but its proof that a fully automated agent could survive, compete, and evolve in a persistent digital world, a concept that continues to define the boundaries of the Screeps sandbox today.

Works cited

1. TooAngel NPC / bot / source code for screeps - GitHub, accessed February 24, 2026, <https://github.com/TooAngel/screeps>
2. TooAngel NPC / bot / source code for screeps - GitHub Pages, accessed February 24, 2026, <http://tooangel.github.io/screeps/>
3. Server-side architecture overview | Screeps Documentation, accessed February 24, 2026, <https://docs.screeps.com/architecture.html>
4. Introduction | Screeps Documentation, accessed February 24, 2026, <https://docs.screeps.com/introduction.html>
5. Screeps - MMO strategy sandbox game for programmers, accessed February 24, 2026, <https://screeps.com/>
6. CPU - Screeps Wiki, accessed February 24, 2026, <https://wiki.screepspl.us/CPU/>
7. Global Control Level - Screeps Wiki, accessed February 24, 2026, https://wiki.screepspl.us/Global_Control_Level/
8. TooAngel | Screeps Forum, accessed February 24, 2026, <https://screeps.com/forum/user/tooangel>
9. Private Server Bot Development - Github-Gist, accessed February 24, 2026, <https://gist.github.com/3510b7f00a07eec901b53164dba7631e>
10. Screeps-AI/README.md at master - GitHub, accessed February 24, 2026, <https://github.com/AmandaRekonwith/Screeps-AI/blob/master/README.md>
11. Tips - Screeps Wiki - Fandom, accessed February 24, 2026, <https://screeps.fandom.com/wiki/Tips>
12. Steam Workshop::[screeps-bot-tooangel] TooAngel bot for screeps server, accessed February 24, 2026, <https://steamcommunity.com/sharedfiles/filedetails/?l=swedish&id=800902233>
13. screeps/doc/Visualization.md at master - GitHub, accessed February 24, 2026, <https://github.com/TooAngel/screeps/blob/master/doc/Visualization.md>
14. screeps/doc/BaseBuilding.md at master - GitHub, accessed February 24, 2026, <https://github.com/TooAngel/screeps/blob/master/doc/BaseBuilding.md>
15. Automating Base Planning in Screeps – A Step-by-Step Guide, accessed February 24, 2026, <https://sy-harabi.github.io/Automating-base-planning-in-screeps/>
16. Question about automated base building/design : r/screeps - Reddit, accessed February 24, 2026, https://www.reddit.com/r/screeps/comments/jpkk55/question_about_automated_base_buildingdesign/
17. ryanrolds/screeps-bot-choreographer - GitHub, accessed February 24, 2026, <https://github.com/ryanrolds/screeps-bot-choreographer>

18. @the-international-screeps-bot/the-international-open-source - NPM, accessed February 24, 2026,
<https://www.npmjs.com/package/%40the-international-screeps-bot%2Fthe-international-open-source>
19. Screeps: First Steps in Automating Room Planning | by Jerroyd Moore | Medium, accessed February 24, 2026,
<https://medium.com/@jerroydmoore/screeps-first-steps-in-automating-room-planning-6fcb2ec89c1a>
20. screeps/doc/Manual.md at master - GitHub, accessed February 24, 2026,
<https://github.com/TooAngel/screeps/blob/master/doc/Manual.md>
21. Great Filters - Screeps Wiki, accessed February 24, 2026,
https://wiki.screepspl.us/Great_Filters/
22. What is your organizational structure for your creeps? : r/screeps - Reddit, accessed February 24, 2026,
https://www.reddit.com/r/screeps/comments/5epp14/what_is_your_organizational_structure_for_your/
23. bencbartlett/Overmind: AI for Screeps, a multiplayer programming strategy game - GitHub, accessed February 24, 2026, <https://github.com/bencbartlett/Overmind>
24. Screeps #1: Overlord overload - Ben Bartlett, accessed February 24, 2026,
<https://bencbartlett.com/blog/screeps-1-overlord-overload/>
25. bonzaiferroni/bonzAI-framework - GitHub, accessed February 24, 2026,
<https://github.com/bonzaiferroni/bonzAI-framework>
26. bucket or larger cpu limit on first tick :: Screeps: Arena General Discussions, accessed February 24, 2026,
<https://steamcommunity.com/app/1137320/discussions/0/3078754800913644025/>
27. How does CPU limit work | Screeps Documentation, accessed February 24, 2026,
<https://docs.screeps.com/cpu-limit.html>
28. Looking For Efficiency Tips : r/screeps - Reddit, accessed February 24, 2026,
https://www.reddit.com/r/screeps/comments/ocdfg8/looking_for_efficiency_tips/
29. Screeps performance concerns - Reddit, accessed February 24, 2026,
https://www.reddit.com/r/screeps/comments/iax8c1/screeps_performance_concerns/
30. Screeps #6: Verifiably refreshed | Ben Bartlett, accessed February 24, 2026,
<https://bencbartlett.com/blog/screeps-6-verifiably-refreshed/>
31. Screeps after one year - Pedantic Orderliness, accessed February 24, 2026,
<https://www.pedanticorderliness.com/posts/screeps>
32. Operating System - Screeps Wiki, accessed February 24, 2026,
https://wiki.screepspl.us/Operating_System/
33. To Code Operating Systems or Not - screeps - Reddit, accessed February 24, 2026,
https://www.reddit.com/r/screeps/comments/790jh0/to_code_operating_systems_or_not/
34. Memory | Screeps Wiki - Fandom, accessed February 24, 2026,
<https://screeps.fandom.com/wiki/Memory>

35. Memory - Screeps Wiki, accessed February 24, 2026,
<https://wiki.screepspl.us/Memory/>
36. How to modify Objects in memory | Screeps Forum, accessed February 24, 2026,
<https://screeps.com/forum/topic/3291/how-to-modify-objects-in-memory>
37. Example for storing and using a path in memory? | Screeps Forum, accessed February 24, 2026,
<https://screeps.com/forum/topic/824/example-for-storing-and-using-a-path-in-memory>
38. Screeps Memory adding how? - Stack Overflow, accessed February 24, 2026,
<https://stackoverflow.com/questions/30324353/screeps-memory-adding-how>
39. All Pages | Screeps Wiki, accessed February 24, 2026,
<https://wiki.screepspl.us/AllPages/>
40. Screeps #6: verifiably refreshed - Reddit, accessed February 24, 2026,
https://www.reddit.com/r/screeps/comments/aby10m/screeps_6_verifiably_refreshed/
41. On the topic of open source code bases | Screeps Forum, accessed February 24, 2026,
<https://screeps.com/forum/topic/2000/on-the-topic-of-open-source-code-bases>
42. bonzaiferroni/bonzAI - GitHub, accessed February 24, 2026,
<https://github.com/bonzaiferroni/bonzAI>
43. Steam Workshop::[screeps-bot-tooangel] TooAngel bot for screeps server, accessed February 24, 2026,
<https://steamcommunity.com/sharedfiles/filedetails/?id=800902233>
44. Private Server Bot Development - Screeps Wiki, accessed February 24, 2026,
https://wiki.screepspl.us/Private_Server_Bot_Development/
45. Strict mode - Screeps Typescript Starter, accessed February 24, 2026,
<https://screepers.gitbook.io/screeps-typescript-starter/in-depth/typescript>
46. Advice on starting out from scratch or with someone else's codebase? : r/screeps - Reddit, accessed February 24, 2026,
https://www.reddit.com/r/screeps/comments/8of5c8/advice_on_starting_out_from_scratch_or_with/
47. So is everyone just using Overmind now?? : r/screeps - Reddit, accessed February 24, 2026,
https://www.reddit.com/r/screeps/comments/cm5o0w/so_is_everyone_just_using_overmind_now/
48. How good are publicly available codebases? : r/screeps - Reddit, accessed February 24, 2026,
https://www.reddit.com/r/screeps/comments/8ddlzd/how_good_are_publicly_available_codebases/
49. Screeps #14: Decision Making - Field Journal, accessed February 24, 2026,
<https://jonwinsley.com/notes/screeps-decision-making>
50. Control | Screeps Documentation, accessed February 24, 2026,
<https://docs.screeps.com/control.html>
51. Help me decide to buy this game answering me these negative steam reviews |

Screeps Forum, accessed February 24, 2026,

<https://screeps.com/forum/topic/2673/help-me-decide-to-buy-this-game-answering-me-these-negative-steam-reviews>

52. Community Communication - Screeps Wiki, accessed February 24, 2026,

https://wiki.screepspl.us/Community_Communication/

53. Screeps #1: The Game Plan | Field Journal, accessed February 24, 2026,

<https://jonwinsley.com/notes/screeps-game-plan>